

Kompendium - Polyformisme

Forholdet mellom superklasse-objekter og subklasse-objekter

Alle variable som er av en eller annen klassetype er, som vi vet, det som kalles referansevariable eller pekere: De kan referere til, eller peke på, et objekt av vedkommende klassetype. For objekter som tilhører samme klassehierarki gjelder følgende viktige regel:

En referansevariabel (peker) av en bestemt klassetype kan også referere til (peke på) objekter av en hvilken som helst subklassetype til vedkommende klasse (direkte eller indirekte subklasse).

Eksempel

Vi tenker oss at vi har følgende klassehierarki:

- Bok
- Fagbok
- Skolebok
- Roman
- NorskRoman
- UtenlandskRoman

Med denne listeoppstillingen så menes det at Fagbok, Skolebok og Roman alle tre er subklasser til Bok. NorskRoman og UtenlandskRoman er i sin tur subklasser til Roman, slik at de dessuten blir indirekte subklasser til Bok.

For en variabel av type Bok vil da følgende gjelde:

Bok b;

b kan nå referere til objekter av type

- Bok
- Fagbok
- Skolebok
- Roman
- NorskRoman
- UtenlandskRoman

- eventuelle nye subklassetyper av de nevnte typer

Det virker jo helt logisk at det er slik det også bør være, for alle de nevnte typer er jo en eller annen type bok! For øvrig er det selvsagt slik at alle de nevnte typene objekter må opprettes ved å bruke **new**-operatoren og en konstruktør for vedkommende klasse før vi kan sette **b** til å referere til dem. Vi kan for eksempel skrive

```
Bok b = new Fagbok( ... ); // med aktuelle konstruktør-parametre
```

Vi kan også si det på den måten at *et objekt av en eller annen subklassetype til en gitt klasse kan behandles **som om** det var et objekt av vedkommende klassetype*. Dette er i samsvar med at det logisk sett er tilfelle at for eksempel alle fagbøker er bøker. Alle norske romaner er også bøker. Men omvendt er det ikke tilfelle at alle bøker er romaner!

Disse omstendighetene har betydning i forbindelse med metoder og metodekall. Følgende to eksempler forsøker å belyse dette.

Eksempel

Vi tenker oss at vi har en metode som er definert i klassen **Bok**:

```
public void bokmetode( Bok b )
{
    < bruk b til et eller annet >
}
```

Ved kall på metoden må vi bruke en aktuell parameter som refererer til et eller annet bokobjekt:

```
bokmetode( a );
```

Men i følge regelen ovenfor har vi mange muligheter for hvilken type bokobjekt parameteren **a** virkelig refererer til: det kan være av typen **Bok** eller en hvilken som helst subklassetype til **Bok** (**Fagbok**, **Skolebok**, **Roman**, **NorskRoman** etc.).

Eksempel

Vi tenker oss så at vi har en metode som er definert i klassen **Roman**:

```
public void romanmetode( Roman r )
{
```

```
< bruk r til et eller annet >
}
```

Kall på metoden:

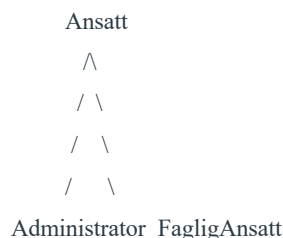
```
romanmetode( c );
```

Den aktuelle parameteren `c` kan være av type `Roman`, `NorskRoman` eller `UtenlandskRoman`. Men `c` kan *ikke uten videre* være av type `Bok`! Dersom `c` er av type `Bok`, kan imidlertid `c` referere til for eksempel et `Roman`-objekt, jamfør det som er sagt foran. Og dersom det er tilfelle, kan `c` konverteres til en referanse som kan brukes som aktuell parameter. For å teste om en referansevariabel refererer til et objekt av en bestemt type, gjør vi bruk av nøkkelordet `instanceof` som i følgende eksempel:

```
if ( c instanceof Roman ) // refererer c til et Roman-objekt?
{
    Roman boka = (Roman) c; //typekonvertering, kan gjøres fordi vi nå
                          //vet at c refererer til et Roman-objekt
    romanmetode( boka ); //typekonverteringen er nødvendig fordi
    .                  //romanmetode krever en parameter av type
    .                  //Roman eller en subklassetype til Roman
    .
}
.
.
.
```

Dynamisk metodebinding — polymorfisme

Tidligere har vi sett på følgende klassehierarki:



I hver av klassene definerte vi en `toString`-metode som returnerte en tekststreng med informasjon om datafeltenes verdier. Dersom vi nå har en variabel av type `Ansatt`:

Ansatt a;

så kan, etter det som er sagt, **a** referere til objekter både av type **Ansatt**, av type **Administrator** og av type **FagligAnsatt** (samt til eventuelt nye subklassetyper som vi måtte finne på å definere).

Det betyr også at dersom vi har en array av type **Ansatt**:

```
int maks = 10; //antallet er uten betydning i denne sammenheng
Ansatt[] personale = new Ansatt[ maks ];
```

så kan vi på hver plass i arrayen sette inn et objekt av en hvilken som helst subklassetype til **Ansatt**, i tillegg til **Ansatt**-objekter. (Egentlig er det referanser vi setter inn. **Ansatt**-arrayen består egentlig av referanser som kan peke på **Ansatt**-objekter, eller på subklasseobjekter.)

Eksempel

Som demonstrasjon av dette skal vi nå lage et enkelt program der vi oppretter en slik array av type **Ansatt** og fyller den med objekter av de tre klassetypene vi har definert i **Ansatt**-hierarkiet. Deretter skal vi gjennomløpe arrayen for å skrive ut den informasjon som er lagret. Det gjør vi ved å gjøre kall på de enkelte objektenes **toString**-metode. Det fine er at det da er unødvendig å foreta noen test på hvilken type objekt referansene **personale[0]**, **personale[1]**, ... refererer til, for å få gjort kall på den riktige **toString**-metoden. Dette finner systemet ut selv. Dette kalles *dynamisk metodebinding* og betyr at det først er når programmet blir kjørt at det avgjøres hvilken metodedefinisjon som skal brukes i et metodekall. Det avgjøres *ikke* under kompileringen av programmet. For under kompileringen er det jo umulig å vite hvilken type objekt i et klassehierarki som en referansevariabel vil komme til å referere til.

Det at en referansevariabel kan referere til forskjellige objekttyper i et klassehierarki og velge ut den riktige av redefinerte metoder for å utføre en oppgave, kalles *polymorfisme*. Det betyr at én og samme identifikator kan opptre i flere forskjellige former eller forkledninger. Resultatet er at vi kan bruke samme instruksjon for å få utført forskjellige handlinger, avhengig av hvilken type objekt som mottar instruksjonen. (Når vi bruker et objekt **obj** for å gjøre kall på en metode definert i klassen som **obj** er en instans av, så sier vi at **obj** mottar en instruksjon.)

I vårt programs utskriftsrutine har vi en **for**-løkke for å gjennomløpe **Ansatt**-arrayen:

```
for ( int i = 0; i < antall; i++ )
    ansatte += personale[ i ].toString() + "\n";
```

Her gjøres det kall på `toString`-metoden til hvert `Ansatt`-objekt som er lagt inn i arrayen. For hvert objekt er det forskjellige instruksjoner som utføres av `toString`-metoden, avhengig av objekttypen: om typen er `Ansatt`, `Administrator` eller `FagligAnsatt`. Og som vi ser av utskriften fra programmet, er det alltid den riktige `toString`-metoden som blir kalt opp.

Klassene `Ansatt`, `FagligAnsatt` og `Administrator` brukes akkurat slik de ble definert i nevnte program i kapittel 9. De to siste av dem er gjengitt nedenfor.

```
1 public class FagligAnsatt extends Ansatt
2 {
3     private String fag;
4     private final int FAGLIGMINIMUM = 45;
5
6     public FagligAnsatt( String n, Dato f, Dato t, String fg )
7     {
8         super( n, f, t );
9         fag = fg;
10        setLønnstrinn( FAGLIGMINIMUM );
11    }
12
13    public String toString()
14    {
15        return super.toString() + ", fagområde: " + fag;
16    }
17
18    public void setLønnstrinn( int trinn )
19    {
20        if ( trinn < FAGLIGMINIMUM )
21            trinn = FAGLIGMINIMUM;
22        super.setLønnstrinn( trinn );
23    }
24 }
```

```
1 public class Administrator extends Ansatt
2 {
3     private String seksjon, stilling;
4     private final int ADMINMINIMUM = 41;
5
```

```

6 public Administrator( String n, Dato f, Dato t, String seksj,
7     String st )
8 {
9     super( n, f, t );
10    seksjon = seksj;
11    stilling = st;
12    setLønnstrinn( ADMINMINIMUM );
13 }
14
15 public String toString()
16 {
17     return super.toString() + ", seksjon: " + seksjon +
18         ", stilling: " + stilling;
19 }
20
21 public void setLønnstrinn( int trinn )
22 {
23     if ( trinn < ADMINMINIMUM )
24         trinn = ADMINMINIMUM;
25     super.setLønnstrinn( trinn );
26 }
27 }

```

Vi skal i det nye programmet opprette de samme ansattobjektene av forskjellig type som vi gjorde i det opprinnelige programmet. Men det vi skal endre på er klassen **Ansatte** som ble brukt i programmet til å inneholde objektene som ble opprettet. Den modifierer vi slik at de fire objektene nå legges inn i en array. Den modifiserte klassen er kalt **Ansatte2** og er gjengitt nedenfor.

```

1 public class Ansatte2
2 {
3     private int maks = 10;
4     private Ansatt[] personale = new Ansatt[ maks ];
5     private int antall = 0; //tellevariabel for arrayen
6
7     public Ansatte2()
8     {
9         personale[ antall++ ] = new Administrator( "Siri Ledestjerne",
10             new Dato( 13, 1, 1983 ), new Dato( 1, 1, 2009 ),
11             "sentraladministrasjon", "direktør" );
12         personale[ antall++ ] = new Administrator( "Per Skrivekarl",
13             new Dato( 17, 5, 1989 ), new Dato( 1, 4, 2010 ),
14             "papirmølla", "sekretær" );

```

```

15  personale[ antall++ ] = new FagligAnsatt( "Tone Regnevik",
16      new Dato( 20, 6, 1985 ), new Dato( 1, 8, 2008 ),
17      "matematikk" );
18  personale[ antall++ ] = new FagligAnsatt( "Else Fortun",
19      new Dato( 29, 2, 1990 ), new Dato( 1, 8, 2006 ), "data" );
20  }
21
22  public String toString()
23  {
24      String ansatte = "Ansatte:\n";
25      for ( int i = 0; i < antall; i++ )
26          ansatte += personale[ i ].toString() + "\n";
27
28      return ansatte;
29  }
30
31  public String getLønnsoversikt()
32  {
33      String tabell = "Navn\t\tLønnstrinn\n";
34      for ( int i = 0; i < antall; i++ )
35          tabell += personale[ i ].getNavn() + "\t\t" +
36              personale[ i ].getLønnstrinn() + "\n";
37
38      return tabell;
39  }
40  }

```

Det i slike tilfelle lurt å ha en tellevariabel for hvor mange objekter som er lagt inn i arrayen. Det er dette som er oppgaven til datafeltet `antall` i klassen vår. Dette har vi blant annet nytte av i `toString`-metoden og i metoden `getLønnsoversikt` der arrayen blir gjennomløpt for å hente ut informasjon fra hvert enkelt objekt.

I programmets `main`-metode må det nå opprettes et objekt av type `Ansatte2` istedenfor av type `Ansatte`, ellers er det ingen endringer i forhold til opprinnelig program.

I programmet merker vi oss spesielt følgende: I `Ansatte2`-klassens `toString`-metode, gjengitt ovenfor, blir `Ansatt`-arrayen `personale` gjennomløpt ved hjelp av en `for`-løkke. Det gjøres kall på objektenes `toString`-metode ved metodekallene

```
personale[ i ].toString()
```

Referansene `personale[ i ]` refererer til forskjellige typer objekter: de to første til `Administrator`-objekter, de to siste til `FagligAnsatt`-objekter. De to klassene har ulikt innhold i sin `toString`-metode. Men av utskriften fra programmet, som er vist nedenfor, ser vi tydelig at det for hvert objekt er den riktige `toString`-metoden som er blitt utført. Vi ser for øvrig at alle ansatte nå er tildelt sin minimumslønn. Dette blir gjort av konstruktøren og det er ikke lagt inn noen instruksjoner i programmet for å endre lønnstrinnet.

Nøkkelordet `final` brukt på metoder og klasser

Vi vet at nøkkelordet `final` blir brukt til å definere konstanter i java, som i eksemplet

```
final int MAXANTALL = 100;
```

Nøkkelordet `final` brukt på en variabel, som i eksemplet, innebærer at variabelen ikke kan tildeles verdi mer enn én gang. Når variabelen først har fått en verdi, kan den ikke seinere endres. Derfor blir variabelen i praksis en konstant. For å markere at det er en konstant, bruker vi bare store bokstaver i navnet på den.

Nøkkelordet `final` kan vi imidlertid også bruke på klasser og metoder. Når vi markerer en metode med `final`, betyr det at ingen subklasse kan redefinere metoden og endre dens virkemåte. Vi har med andre ord skrevet dens `final`, det vil si endelige versjon. Hele klasser kan også markeres som `final`:

```
final class IkkeUtvidbar
{
    ...
}
```

En `final` klasse er det ikke tillatt å definere subklasser av. Alle dens metoder blir derfor implisitt `final` metoder.

Hva er så hensikten med å markere en metode som `final`? Den viktigste har med sikkerhet å gjøre: Alle som bruker metoden kan være sikre på at dens virkemåte ikke endres, uansett hvilken type (subklasse)objekt som brukes for å kalle den opp. Dersom en klasse er `final`, kan ingen definere en subklasse som bryter med de garantier som klassen er ment å oppfylle. Dette kan for eksempel være aktuelt med kode som brukes ved innlesing og validering av passord.

På den andre side innebærer bruk av `final` en streng restriksjon på bruken av en klasse og begrenser dens fleksibilitet. Det bør derfor ikke brukes uten at vi har en viktig grunn til det. Istedenfor å markere en hel klasse som `final`, kan vi i mange tilfeller oppnå den nødvendige sikkerhet ved å markere hver enkelt metode i klassen med `final`. Da kan vi stole på virkemåten til disse metodene, og samtidig åpne for muligheten til å utvide klassen med mer funksjonalitet uten å kunne redefinere de metodene som arves. Pass imidlertid på at metoder som er `final` lages slik at de bare er avhengige av datafelter som enten er `final` eller `private`. Ellers vil en subklasse kunne endre virkemåten til metodene ved å endre på disse datafeltene.

En annen hensikt med bruk av `final`, kan være optimalisering, det vil si minimalisering av kjøretid. Når det gjøres kall på en metode som ikke er `final`, vil Javas kjøresystem først bestemme klassetypen til det objektet som har gjort kall på metoden, deretter knytte kallet til den riktige implementasjonen av vedkommende metode, og så utføre koden som denne inneholder. For en metode som er `final`, kan det allerede under kompileringen avgjøres hvilken kode som skal utføres, og metodekallet kan rett og slett erstattes ved å putte inn vedkommende kode på kallstedet. Tilsvarende kan gjøres med metoder som er `private` og `static`, siden de heller ikke kan redefineres i subklasser. For `final` klasser er det også enkelte typer sjekking som kan foretas allerede under kompilering, sjekking som for andre klasser først kan foretas under kjøring.

I Javas klassebibliotek er blant annet `String`-klassen markert som `final`.

Copyright © Kjetil Grønning og Eva Hadler Vihovde